MODULE 31 · DOMAIN 4 - OPERATING, MONITORING, SECURING

Observability and troubleshooting

CloudWatch, Logs Insights, CloudTrail, X-Ray, and diagnosing what went wrong.

MODULE 31 of 43 DOMAIN 4 - Monitoring WEIGHT 24%

admin@sthithapragna.com

Is it healthy? What happened? Who did it?
Different questions, different services.

Which service answers which question

QUESTION	WHERE TO LOOK
Is the endpoint healthy right now?	CloudWatch metrics
What did the container actually print?	CloudWatch Logs
Why did this one request fail?	CloudWatch Logs, and the error body
Where did the time go across services?	AWS X-Ray
Who created, changed or deleted this?	AWS CloudTrail
How do I find one line among millions?	CloudWatch Logs Insights

ON THE EXAM

CloudTrail records API calls, not application behaviour. It will never tell you why a model returned a bad prediction.

SageMaker AI CloudWatch namespaces

- **AWS/SageMaker** holds request metrics from InvokeEndpoint, and model loading metrics.
- **/aws/sagemaker/TrainingJobs**, **/TransformJobs** and **/Endpoints** hold **instance** metrics.
- **aws/sagemaker/InferenceComponents** holds metrics for component-based endpoints. Module 21.
- Looking in the wrong namespace is why people conclude a metric does not exist.
- Endpoint metric dimensions are **EndpointName** and **VariantName**.

> ON THE EXAM
>
> Request metrics and instance metrics are in different namespaces. Invocation counts and CPU utilisation are not in the same place.

The request metrics that matter

METRIC	WHAT IT TELLS YOU
Invocations	Traffic. Zero when you expected some is itself a finding
Invocation4XXErrors	The caller sent something the model rejected
Invocation5XXErrors	The model or container failed
ModelLatency	Time inside the container. Your model's own cost
OverheadLatency	Time SageMaker AI added around it
ContainerLatency	Per container. Appears only for inference pipelines

ON THE EXAM

ModelLatency against OverheadLatency is the first split: is the model slow, or is something around it slow?

CPUUtilization can legitimately exceed 100 percent

- **CPUUtilization ranges from 0 to 100 percent multiplied by the number of CPUs.**
- On a four-CPU instance, it can read up to **400 percent**, and 380 percent is healthy saturation.
- So divide by the core count before deciding anything is wrong.
- An alarm set at 90 percent on a multi-core instance fires almost immediately and permanently.
- Memory and disk utilisation behave normally, as a percentage of the total.

> ON THE EXAM
>
> This single fact causes more false alarms than anything else in SageMaker monitoring. Set thresholds against cores times 100.

Everything printed lands in CloudWatch Logs

- **Anything a container writes to stdout or stderr** goes to CloudWatch Logs automatically.
- That covers training, processing, transform, endpoint, compilation and notebook containers.
- So a print statement in your training script is a log line, with no configuration.
- Training logs go to **/aws/sagemaker/TrainingJobs**, with **one log stream per node**.
- The stream name begins with the value you gave as TrainingJobName.

> ON THE EXAM
>
> One stream per node matters for distributed training: an error on rank 3 is in rank 3's stream, not the first one you open.

Metric definitions on a training job

- A training job takes **metric definitions**: a name, and a **regular expression** over the log output.
- SageMaker AI scrapes matching lines and publishes them as **CloudWatch metrics**.
- That is how a custom training script's loss curve becomes a graph you can watch live.
- **This is also how tuning reads your objective metric**, from module 15. A wrong regex breaks tuning silently.
- Built-in algorithms define theirs already. Custom scripts must declare them.

> ON THE EXAM
>
> Module 15's tuning depends on this. If the regex does not match, the tuning job sees no metric and cannot optimise anything.

Metric filters

- A **metric filter** on a log group turns matching log lines into a CloudWatch metric.
- So 'count of lines containing OutOfMemoryError' becomes a metric you can alarm on.
- Useful for errors your application knows about and AWS does not.
- Works on any log group, including endpoint container logs.
- Filters apply to **new** log events. They do not retroactively scan history.

> ON THE EXAM
>
> This is how an application-specific failure becomes an alarm, rather than something someone notices while reading logs.

Alarms and composite alarms

- An alarm watches a metric against a threshold over a number of periods.
- **Evaluation periods matter:** one bad minute is noise, five consecutive bad minutes is a signal.
- **Composite alarms** combine several alarms, so you page once rather than five times.
- Treat missing data deliberately: missing can mean healthy, or can mean nothing is running.
- An endpoint with no traffic produces no metrics, so a naive alarm may sit in INSUFFICIENT_DATA.

> ON THE EXAM
>
> The missing-data setting is the one people get wrong. 'No metrics' and 'metrics look fine' are very different states.

If a job failed and there are **no logs at all**, the failure happened before training started.

Failures before training starts

- **The wrong training image.** A bad ECR URI, or an image in another Region.
- **The wrong S3 location.** The input path does not exist, or is empty.
- **Permissions.** The execution role cannot read the bucket, or cannot pull the image.
- **Networking.** A VPC configuration with no route to S3 or ECR. Module 35.
- All four fail identically from the outside: the job fails, and nothing was logged.

> ON THE EXAM
>
> Check FailureReason first. It usually names which of these it was, and it is one API call away.

SecondaryStatus tells you the phase

PHASE AT FAILURE	WHERE TO LOOK
Starting	Instance provisioning, quotas, or the container failing to start
Downloading	Input data: the S3 path, permissions, or the data itself
Training	Your code, the hyperparameters, or the data's content
Uploading	The output path, or permissions to write to it

ON THE EXAM

SecondaryStatusTransitions gives the full timeline with timestamps, so you can see how long each phase took before it failed.

Training failures that are really data failures

SYMPTOM	USUAL CAUSE
Fails immediately in Training phase	Wrong content type, or a header row the algorithm did not expect
Fails partway through an epoch	A malformed record, or a null the code does not handle
Out of memory on a small dataset	Loading everything into memory. Use Pipe or FastFile mode
Runs but learns nothing	Labels misaligned, or the target column in the wrong position
Works on a sample, fails on full data	Scale, or a value only present in the full set

ON THE EXAM

Module 13's rule again: built-in algorithms expect a specific format, and most 'algorithm errors' are format errors.

FailureReason and the describe call

- **describe_training_job** returns **FailureReason**, a description of what went wrong.
- That is the first thing to read, before opening any logs. It frequently names the cause outright.
- The same pattern applies to processing, transform and endpoint resources.
- Some errors appear **only** in the logs, such as a bad hyperparameter rejected by the algorithm.
- So the order is: FailureReason, then SecondaryStatus, then the logs.

> ON THE EXAM
>
> Module 13's common errors live here: a misspelled hyperparameter shows as a validation error in the log, not in FailureReason.

ModelError, and the status hidden inside it

- **Every non-2xx response from an endpoint arrives at the client as ModelError, HTTP 424.**
- The real status is in **OriginalStatusCode**, and the real message in **OriginalMessage**.
- So a 400 from your container and a 500 from your container both look like 424 at first.
- Client code that only logs the outer error discards the useful half of the information.
- The container's own log line for that request usually explains it fully.

> ON THE EXAM
>
> 424 is not the error. It is the envelope. The error is inside it, and most teams discover this during their first incident.

Symptom to first check

SYMPTOM	FIRST THING TO CHECK
Endpoint stuck Creating, then Failed	Service quota for that instance type. Read the failure reason
All invocations return 424	OriginalStatusCode and the container logs
4XX errors only on some requests	Input format. Content type, schema, or a field the model rejects
5XX errors under load only	Container crashing, memory pressure, or worker exhaustion
High ModelLatency, low OverheadLatency	The model itself. Module 22's optimisation levers
High OverheadLatency	Payload size, serialisation, or model loading
No metrics at all	Usually no traffic. Metrics appear with consistent invocations

ON THE EXAM

The first row is the commonest deployment failure in a new account, and it has nothing to do with your model.

Troubleshooting a pipeline execution

WHERE IT FAILED	WHAT TO CHECK
A step failed	That step's underlying job: its FailureReason and its logs
ConditionStep could not evaluate	The property file path against the report's actual structure
Steps ran in the wrong order	A missing data dependency, or a needed add_depends_on
The pipeline will not start	Execution role permissions, or a parameter that failed validation
Everything passed but nothing registered	The condition evaluated false. Read the metric it read

ON THE EXAM

A pipeline failure is always a step failure, and a step failure is always an ordinary job failure. Drill down rather than sideways.

Diagnosing permission failures

- **AccessDenied** messages name the **principal**, the **action** and the **resource**. Read all three.
- The principal is usually a **role**, and often not the one you assumed was being used.
- Remember there are **several roles**: your user, the execution role, and each job's own role.
- A job failing in the Downloading phase with no logs is very often a bucket permission.
- **IAM policy simulator** and CloudTrail both help confirm which policy denied it.

> ON THE EXAM
>
> Read the principal in the message before editing any policy. Half of permission debugging is discovering which identity was actually acting.

Diagnosing quotas and throttling

- **ResourceLimitExceeded** on a job means an account quota, not a bug in your code.
- Accelerated instance quotas are frequently **zero by default** in a new account. Modules 14 and 20.
- **ThrottlingException** means an API rate limit. Back off and retry, then request an increase.
- An endpoint that will not scale past a number is hitting an instance quota, not its policy maximum.
- Service Quotas shows the applied value and the usage against it, per Region.

> ON THE EXAM
>
> Quota errors look like failures and are actually paperwork. Knowing that saves a day of debugging code that is fine.

CloudWatch Logs Insights

- A query language over log groups, for finding and aggregating rather than scrolling.
- **filter** to narrow, **stats** to aggregate, **sort** and **limit** to make it readable.
- Query across **several log groups at once**, which matters with one stream per node.
- Useful patterns: count errors by type, find the slowest requests, extract a field and aggregate it.
- Queries scan data and are billed by the volume scanned. Narrow the time range first.

> ON THE EXAM
>
> Scrolling a log group is a last resort. Anything you would scroll for, a query answers in seconds and across every node.

Subscription filters

- A **subscription filter** streams matching log events out of CloudWatch in near real time.
- Destinations include **Kinesis Data Streams, Firehose and Lambda**.
- That is how logs reach a central SIEM, a data lake, or a third-party observability platform.
- Filter at the subscription, so you stream what you need rather than everything.
- Per Region, per log group. Module 30's fragmentation point applies here too.

> ON THE EXAM
>
> This is the export path. Teams that already run one observability platform use it rather than maintaining two consoles.

Cross-account observability

- **CloudWatch cross-account observability** designates a **monitoring account** that can see others.
- That fits module 28's structure, where development, staging and production are separate accounts.
- Metrics, logs and traces can all be shared into the monitoring account.
- Without it, investigating a production incident means assuming a role in another account each time.
- Set it up before an incident. Doing it under pressure is slow and error-prone.

> ON THE EXAM
>
> The multi-account boundary that protects production also gets in the way of watching it. This is the reconciliation.

CloudTrail

- Records **console actions and API or CLI calls** as events, across AWS services.
- Each event carries **identity information**: root, an IAM user, temporary role credentials, or a service.
- This is how you answer who deleted the endpoint, who changed the role, who approved the model.
- It records the **call**, not the effect. It will not tell you why a prediction was wrong.
- Management events are on by default. Data events generally are not.

> ON THE EXAM
>
> CloudTrail answers questions about people and automation. CloudWatch answers questions about behaviour. Neither substitutes for the other.

CloudTrail and the Studio execution role

- **By default, CloudTrail logs the Studio execution role name**, not the individual user.
- So several data scientists sharing a role appear in the audit trail as the same actor.
- That is fine for knowing what happened, and useless for knowing who did it.
- The fix is per-user roles or user profiles that carry identity through to the trail.
- Worth discovering before an audit asks, rather than during one.

> ON THE EXAM
>
> A shared execution role is convenient to set up and produces an audit trail that cannot attribute anything to a person.

Auditing generative workloads

- Bedrock API calls appear in **CloudTrail** like any other service.
- So you can see who invoked which model, who changed a guardrail, and who created a knowledge base.
- CloudTrail records the **call**. The prompt and response are in **invocation logs**, from module 30.
- Those two together answer both who and what, and neither does it alone.
- Configuration changes matter here: a guardrail weakened is an audit event worth alarming on.

> ON THE EXAM
>
> A weakened guardrail or a deleted policy is a CloudTrail event. That is a more useful alarm than most application metrics.

AWS X-Ray

- Distributed tracing: follows one request across several services and shows where time went.
- Right when an ML call is **one hop in a larger application**: API Gateway, Lambda, then an endpoint.
- It answers whether the model is slow, or something around the model is slow.
- Requires instrumentation, so it is a design decision rather than a switch.
- Module 30's prompt tracing is the generative equivalent, for components rather than services.

> ON THE EXAM
>
> ModelLatency tells you the model's time. X-Ray tells you the user's time, which is usually a much larger number.

Studio and notebook troubleshooting

- Notebook and lifecycle configuration output also goes to **CloudWatch Logs**.
- A **lifecycle configuration that fails** prevents the app from starting, and the reason is in its log.
- Lifecycle configurations have a **time limit**. A slow install can exceed it and fail the start.
- Kernel dying during work is usually memory: the instance is too small for the data loaded.
- Studio runs on an instance you are paying for. An idle app is module 32's problem.

> ON THE EXAM
>
> The commonest Studio failure is a lifecycle script, and its log is in the same place as everything else.

Container Insights and resource metrics

- **Container Insights** gives container-level CPU, GPU, memory, network and disk metrics.
- That matters for training clusters, where the question is which node or which GPU is the problem.
- Named in the Well-Architected guidance for enhanced ML infrastructure monitoring.
- It covers CPU, GPU, Trainium, EFA and file system metrics at container level.
- More detail means more ingestion cost. Enable it where the detail will actually be used.

ON THE EXAM

Module 14's distributed training question 'is the GPU the bottleneck or the data pipeline' is answered at this level of detail.

Retention, and what it costs

ITEM	BEHAVIOUR
CloudWatch metrics	Retained 15 months, at decreasing resolution over time
CloudWatch Logs	Retained until you say otherwise. Set a period deliberately
Log ingestion	Charged per GB ingested. The dominant log cost
Logs Insights queries	Charged by data scanned. Narrow the time range
CloudTrail management events	A 90-day event history is available without a trail
X-Ray traces	Charged per trace recorded and retrieved

ON THE EXAM

Logs default to never expiring. That is a bill and a data retention obligation accumulating quietly together.

Dashboards for an ML platform

- One dashboard per **system**, not per service. Nobody wants to correlate six tabs during an incident.
- Put **training, endpoint and pipeline health** on one board if they serve one product.
- Include the latency split, error rates, saturation, and the last pipeline execution status.
- Annotate thresholds on the graphs, so a reader can see where normal ends.
- A dashboard built during an incident is a dashboard built too late.

> ON THE EXAM
>
> The test of a dashboard: can somebody who did not build the system tell whether it is healthy in ten seconds?

How to diagnose, in order

- **1. Read the resource.** FailureReason, status, SecondaryStatus. One API call.
- **2. Check whether logs exist at all.** Absence is a diagnosis, not a dead end.
- **3. Read the logs**, on the right node, around the right timestamp.
- **4. Check the metrics** for saturation, errors and the latency split.
- **5. Check CloudTrail** if something changed that nobody mentioned.

> ON THE EXAM
>
> Most people start at step three. Steps one and two are faster and frequently finish the job on their own.

Third-party observability

- Many organisations already run a platform for logs, metrics and traces across everything they own.
- The route in is **subscription filters** for logs and **metric streams** for metrics.
- OpenTelemetry, through **ADOT**, is the portable instrumentation layer. Modules 27 and 30.
- The benefit is one place to look, correlated with the rest of the estate.
- For the exam, answer with the AWS-native service unless a scenario names an existing platform.

> ON THE EXAM
>
> Same pattern as MWAA in module 28: existing investment is the legitimate reason, and it should be stated in the scenario.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Job failed, no logs at all	Failure before training started. Image, path, permissions or VPC
CPUUtilization alarm always firing	Threshold not multiplied by core count
Client sees only HTTP 424	ModelError envelope. Read OriginalStatusCode
Endpoint Creating then Failed	Instance quota. Read the endpoint failure reason
Metric appears to not exist	Wrong namespace. Request and instance metrics are separate
No endpoint metrics	No consistent traffic yet
Audit cannot identify who acted	Shared Studio execution role in CloudTrail
Logs bill rising steadily	No retention period set on the log groups

ON THE EXAM

Rows one and two are the two most useful facts in this module, and neither is about a tool. Both are about interpretation.

Absence is evidence. No logs, no metrics and no traffic each tell you something specific.

> A training job fails within ninety seconds of starting. The team opens CloudWatch, navigates to /aws/sagemaker/TrainingJobs, and finds no log stream for the job at all.
>
> They conclude CloudWatch logging is broken and open a support case.

What is actually happening?

A. Log delivery is delayed. The streams will appear within fifteen minutes

B. Logging must be enabled per training job, and it was not

C. The logs are in the /aws/sagemaker/Endpoints namespace instead

D. The failure happened before training started: image, S3 path, permissions or VPC

◆ PAUSE AND DECIDE

What is actually happening?

A Log delivery is delayed. The streams will appear within fifteen minutes

Delivery is near real time. A ninety-second failure with no stream at all is not a delivery delay.

B Logging must be enabled per training job, and it was not

Wrong: anything a container writes to stdout or stderr goes to CloudWatch Logs automatically, with no configuration.

C The logs are in the /aws/sagemaker/Endpoints namespace instead

That namespace holds endpoint instance metrics. A training job does not write there.

D The failure happened before training started: image, S3 path, permissions or VPC

AWS documents exactly this: no logs means the error occurred before the start of training. The container never printed anything because it never ran.

THE SIGNAL

The absence is the diagnosis. Reading FailureReason next will usually name which of the four it was.

> An application calling a SageMaker AI endpoint logs every failure as 'ModelError: 424'. The team cannot tell whether requests are being rejected as malformed or whether the container is crashing.
>
> CloudWatch shows both 4XX and 5XX invocation errors.

What should they do?

A. Add a retry with backoff, since 424 indicates a transient failure

B. Read OriginalStatusCode and OriginalMessage from the error, and the container log line

C. Split the endpoint into two, one per request type, to separate the errors

D. Enable data capture, so the failing payloads are recorded

◆ PAUSE AND DECIDE

What should they do?

A. Add a retry with backoff, since 424 indicates a transient failure

424 is the envelope every non-2xx arrives in, not a transient signal. Retrying a malformed request fails identically every time.

B. Read OriginalStatusCode and OriginalMessage from the error, and the container log line

The real status and message are inside the ModelError. Combined with the container's log line for that request, this identifies the cause directly.

C. Split the endpoint into two, one per request type, to separate the errors

Infrastructure work to infer something the error already contains. It also doubles the endpoints to run and monitor.

D. Enable data capture, so the failing payloads are recorded

Useful for later analysis, and it does not explain why the container rejected them. The error already carries that.

THE SIGNAL

424 is an envelope, not a diagnosis. The information the team says it lacks is already in the response they are discarding.

> A production endpoint was deleted overnight. Nobody admits to deleting it. The team finds the DeleteEndpoint call in CloudTrail, made through the Studio execution role shared by the data science team.
>
> Compliance asks which individual performed the action.

What should you tell them, and what should change?

A CloudTrail logged the shared execution role, so the individual cannot be identified. Move to per-user roles

B Enable CloudTrail data events, which record the individual user

C Check CloudWatch Logs, which record the Studio user for each action

D The information is in X-Ray traces for the Studio session

◆ PAUSE AND DECIDE

What should you tell them, and what should change?

A CloudTrail logged the shared execution role, so the individual cannot be identified. Move to per-user roles

By default CloudTrail logs the Studio execution role name, not the user. With a shared role, every member appears identically. Per-user roles fix it going forward.

B Enable CloudTrail data events, which record the individual user

Data events cover object-level operations on data resources. They do not add user identity that the role already obscured.

C Check CloudWatch Logs, which record the Studio user for each action

CloudWatch Logs hold container output, not a record of who invoked an API. That is CloudTrail's job.

D The information is in X-Ray traces for the Studio session

X-Ray traces application requests across services. It does not audit console or API identity.

THE SIGNAL

Note the honest half of the answer: the past cannot be recovered, only the future fixed. That is worth saying plainly to compliance.

> An engineer sets a CloudWatch alarm on CPUUtilization at 90 percent for a training job running on an instance with 8 vCPUs. The alarm fires within two minutes and stays in ALARM for the whole job.
>
> The job completes successfully and on schedule. GPU utilisation is high and memory is comfortable.

What is wrong?

A. The instance is undersized. Move to a larger instance type

B. The alarm period is too short. Increase it to fifteen minutes

C. CPUUtilization is 100 percent times the core count, so 90 is low for 8 vCPUs

D. The training script is single-threaded and saturating one core

◆ PAUSE AND DECIDE

What is wrong?

A The instance is undersized. Move to a larger instance type

The job completes on schedule with high GPU utilisation and comfortable memory. Nothing indicates a resource problem.

B The alarm period is too short. Increase it to fifteen minutes

The metric genuinely sits above 90 for the whole job. A longer period delays the same false alarm rather than preventing it.

C CPUUtilization is 100 percent times the core count, so 90 is low for 8 vCPUs

The metric ranges 0 to 800 percent on 8 vCPUs. A threshold of 90 is about 11 percent of capacity, so it fires immediately and permanently.

D The training script is single-threaded and saturating one core

Would show CPUUtilization near 100 and no higher. It also would not explain high GPU utilisation and an on-schedule completion.

THE SIGNAL

The alarm is wrong, not the job. On an 8 vCPU instance a sensible threshold is a proportion of 800, not of 100.

What to take from this module

- **No logs at all means the failure happened before training started.** The best diagnostic here.
- **CPUUtilization is 100 percent times the core count.** Set thresholds accordingly.
- **Every endpoint error arrives as ModelError 424.** Read OriginalStatusCode.
- **FailureReason, then SecondaryStatus, then logs.** In that order.
- **CloudTrail answers who. CloudWatch answers what.** And Studio's shared role obscures who.

> ON THE EXAM
>
> Module 32 turns to cost: Savings Plans, Spot, right-sizing, and the idle endpoint problem.